SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I Deva Dharshini(192472350) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Deva Dharshini

Annexure A

SCENARIO-BASED TASK

1.Scenario: Parsing Table Conflict in LL(1) While designing an LL(1) parser, a student constructs a parsing table and finds multiple entries in a single cell for a non-terminal and input symbol combination. This leads to confusion during parsing. Analyze how the syntax analyzer uses FIRST and FOLLOW sets in this process, identify the reason for the conflict, and explain how it affects deterministic parsing. Conclude by suggesting what decision should be taken to modify the grammar so that it becomes suitable for LL(1) parsing.

Introduction

Syntax analysis is a phase of a compiler that checks whether the input tokens follow the grammar rules of a programming language. An LL(1) parser is a top-down parser that reads input from Left to right, produces a Leftmost derivation, and uses one lookahead symbol to select a production.

It uses FIRST and FOLLOW sets to construct a predictive parsing table. If multiple productions appear in the same table cell, a parsing conflict occurs, indicating that the grammar is not LL(1) in its current form.

1. Role of FIRST and FOLLOW Sets

FIRST Set: The FIRST set of a grammar symbol contains the terminals that can appear first in strings derived from that symbol. If the symbol can derive the empty string, ε is also included.

For example:

A → aB | bC

FIRST(A) = {a, b}

FOLLOW Set: The FOLLOW set of a non-terminal contains the terminals that can appear immediately after it. The end marker $ is included in the FOLLOW set of the start symbol.

For example:

S → AB

A → a

B → b

Here, FOLLOW(A) = {b} because A is followed by B, which begins with b.

FIRST and FOLLOW sets help the parser determine which production should be selected for a given input symbol.

2. Identification of the Parsing Conflict

A predictive parsing table has non-terminals as rows and terminals as columns. Each cell indicates the production to apply for the corresponding non-terminal and lookahead symbol.

Consider the grammar:

A → aB | aC

Both productions begin with the terminal a.

Therefore:

FIRST(aB) = {a}

FIRST(aC) = {a}

Both productions must be entered into the same table cell, M[A, a]. The parser cannot decide which production to select using only one lookahead symbol. This is called a FIRST/FIRST conflict.

A FIRST/FOLLOW conflict can occur when a non-terminal has an ε-production and a terminal in its FOLLOW set also belongs to the FIRST set of another alternative production. This can cause multiple productions to occupy the same table cell.

3. Effect on Deterministic Parsing

An LL(1) parser must select exactly one production using the current non-terminal and one lookahead symbol. Multiple entries create uncertainty in production selection.

The main effects are:

1. The parser cannot determine the correct production uniquely.
2. Predictive parsing may fail because it requires a single production for each table entry.
3. Selecting an incorrect production may lead to syntax errors.
4. Backtracking may be required to try alternative productions, reducing parsing efficiency.

Therefore, a grammar that produces conflicting table entries cannot be used directly for deterministic LL(1) parsing.

4. Methods to Resolve the Conflict

Left Factoring: This technique removes common prefixes from alternative productions.

Original grammar:

A → aB | aC

Modified grammar:

A → aA'

A' → B | C

This allows the parser to recognize the common prefix before selecting between the remaining alternatives. The grammar must still be checked for other conflicts.

Elimination of Left Recursion: Left recursion occurs when a non-terminal derives a string beginning with itself.

For example:

$E \rightarrow E + T \mid T$

It can be rewritten as:

$E \rightarrow T\ E'$

$E' \rightarrow + T\ E' \mid \varepsilon$

This transformation makes the grammar suitable for predictive parsing with respect to left recursion, although other LL(1) conditions must also be satisfied.

If the grammar is ambiguous, it should be rewritten to remove the ambiguity where possible.

Methods to Eliminate the Parsing Conflict

Add a table summarizing the solutions.

Type of Problem	**Technique**	**Purpose**
Common prefixes	Left factoring	Removes common prefixes from alternatives
Left recursion	Left recursion elimination	Makes the grammar suitable for predictive parsing
Ambiguous productions	Grammar rewriting	Removes ambiguity where possible
Multiple parsing table entries	Recalculate FIRST and FOLLOW sets	Checks whether the conflict has been resolved

5. Decision and Solution

The student should identify the source of the conflict by examining the FIRST and FOLLOW sets. Left factoring should be applied when productions have common prefixes, while left recursion should be eliminated when present. Ambiguous grammar rules should also be revised where necessary.

After modifying the grammar, the FIRST and FOLLOW sets must be recalculated, and the parsing table must be constructed again. Every table cell should contain at most one production.

Conclusion

FIRST and FOLLOW sets are essential for constructing an LL(1) predictive parsing table. Multiple productions in the same cell indicate that the parser cannot make a unique decision using one lookahead symbol. Therefore, the grammar must be modified using suitable techniques such as left factoring, left recursion elimination, or ambiguity removal. This ensures conflict-free table entries and supports deterministic syntax analysis.

2.Scenario: Incorrect Parse Tree Generation A syntax analyzer generates a parse tree for the expression id + id - id, but the evaluation gives incorrect results due to improper grouping of operators. Analyze how the syntax analyzer constructs the parse tree, identify the issue related to associativity, and explain how grammar rules influence the structure of the tree. Finally, discuss what decision should be taken to enforce correct associativity and ensure accurate parsing.

Introduction

A parse tree is a tree representation of the syntactic structure of an expression according to the grammar rules of a programming language. The syntax analyzer constructs the parse tree by applying grammar productions to the input tokens.

In the given scenario, the expression id + id - id produces an incorrect evaluation because the operators are grouped improperly. This problem is related to operator associativity and the grammar rules used to construct the parse tree.

1. Construction of the Parse Tree

The syntax analyzer receives tokens from the lexical analyzer and checks whether they follow the grammar. It applies suitable production rules to construct a parse tree.

Consider the expression:

id + id - id

This expression contains two operators: addition (+) and subtraction (-).

The expression can be grouped in two ways:

Left-associative grouping:

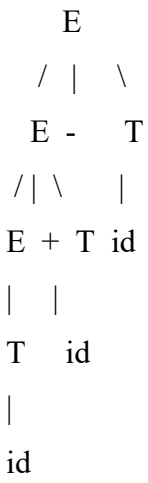
(id + id) - id

Right-associative grouping:

id + (id - id)

These groupings produce different parse trees and can produce different results depending on the values of the identifiers.

Parse Tree:



2. Identification of the Associativity Issue

Associativity determines how operators with the same precedence are grouped when they appear consecutively in an expression.

Addition and subtraction are normally left-associative. Therefore, the expression:

id + id - id

must be interpreted as:

(id + id) - id

For example, if the identifiers have values 10, 5, and 2, respectively:

Left-associative evaluation:

(10 + 5) - 2 = 13

Right-associative evaluation:

10 + (5 - 2) = 13

In this example, both results happen to be equal. However, this is not always the case for other operators, particularly subtraction and division. For example:

20 - 10 - 5

Correct left-associative evaluation:

(20 - 10) - 5 = 5

Right-associative evaluation:

20 - (10 - 5) = 15

Therefore, the grammar must ensure that operators are grouped according to the intended associativity.

3. Influence of Grammar Rules

Consider the following grammar:

E → E + T | E - T | T

T → id

This grammar represents left-associative addition and subtraction when used with a suitable parser.

The recursive productions place E on the left side of the operator. Consequently, the parser can construct the expression from left to right.

However, consider the grammar:

E → T + E | T - E | T

Here, the recursive non-terminal appears on the right side. This structure can produce right-associated expressions.

Thus, the placement of recursive non-terminals in grammar productions influences the structure of the parse tree.

Feature	**Left-Associative Grammar**	**Right-Associative Grammar**
General grouping	(a - b) - c	a - (b - c)
Recursive structure	Recursion on the left	Recursion on the right
Example grammar	E → E - T | T	E → T - E | T
Evaluation order	Groups from left to right	Groups from right to left
Typical use	Addition and subtraction	Assignment operators in many programming languages

4. Decision and Solution

The grammar must be designed to enforce the required associativity.

For a parser that supports left recursion, the following grammar can be used:

E → E + T | E - T | T

T → id

For an LL(1) predictive parser, left recursion must be eliminated. The grammar can be rewritten as:

E → T E'

E' → + T E' | - T E' | ε

T → id

This grammar removes left recursion and allows predictive parsing. However, the parser must still preserve left associativity when constructing the expression structure or evaluating the operators. This can be achieved by processing the operators from left to right using suitable semantic actions or an equivalent left-associative syntax-tree construction method.

Conclusion

Incorrect parse tree generation can occur when the grammar does not enforce the intended operator associativity. Addition and subtraction are normally left-associative, so the expression id + id - id should be

grouped as (id + id) - id. Therefore, the grammar and parsing procedure must be designed to preserve the correct associativity. This ensures that the syntax analyzer constructs the appropriate expression structure and supports accurate evaluation.

3.Scenario: Invalid Expression Handling A syntax analyzer in a compiler receives the token sequence corresponding to the expression a + * b from the lexical analyzer. Although all tokens are valid individually, the parser fails to generate a parse tree and reports an error. In this situation, analyze how the syntax analyzer identifies the error based on grammar rules, determine the key issue in the token arrangement, and explain how parsing techniques such as predictive or shift-reduce parsing handle such cases. Further, discuss what decision should be taken to improve error detection and recovery while maintaining clarity in parsing.

Introduction

Syntax analysis is a phase of a compiler that checks whether the sequence of tokens produced by the lexical analyzer follows the grammar rules of a programming language. It identifies errors in the arrangement of tokens and determines whether an expression is syntactically valid.

In the given scenario, the syntax analyzer receives the expression a + * b. Although all the individual tokens are valid, their arrangement violates the grammar rules. Therefore, the parser cannot generate a valid parse tree and reports a syntax error.

1. Identification of the Syntax Error

Consider the following grammar for arithmetic expressions:

E → E + T | T

T → T * F | F

F → id | (E)

Here, E represents an expression, T represents a term, F represents a factor, and id represents an identifier.

The input expression is:

a + * b

After reading a +, the parser expects a valid term or operand. However, the next token is the multiplication operator *.

A multiplication operator must appear between two operands. Since no operand appears between + and *, the expression violates the grammar rules.

Therefore, the syntax analyzer identifies * as an unexpected token.

input expression: a + * b

Token	Expected Syntax	Actual Token	Result
a	Identifier	a	Valid
+	Addition operator	+	Valid
After +	Identifier or opening parenthesis	*	Syntax error
b	Identifier	b	Cannot complete the invalid expression without recovery

2. Handling Using Predictive Parsing

A predictive parser uses a parsing table to select the appropriate production based on the current non-terminal and lookahead symbol.

When parsing a + * b, the parser first recognizes a as a valid identifier and processes the addition operator. After +, it expects a valid term, which should begin with an identifier or an opening parenthesis.

Instead, the parser encounters *. Since this token cannot begin a valid term or factor, the corresponding parsing table entry indicates an error.

The parser reports a syntax error because the input does not match the expected grammar.

3. Handling Using Shift-Reduce Parsing

A shift-reduce parser uses a stack and an input buffer to analyze the expression. It shifts input tokens onto the stack and reduces recognized sequences according to grammar productions.

While processing a + * b, the parser can recognize a as a valid factor and shift the + operator. However, the multiplication operator appears before the next operand.

The parser cannot reduce the token sequence to a valid expression because the grammar requires an operand or factor in that position.

Therefore, the shift-reduce parser detects the invalid arrangement and reports a syntax error.

4. Error Detection and Recovery

The syntax analyzer should identify the unexpected token and provide a meaningful error message. Suitable error-recovery techniques include:

1. Panic-Mode Recovery: The parser skips input tokens until it reaches a synchronizing symbol, such as a semicolon or closing parenthesis, and then continues parsing.

2. Phrase-Level Recovery: The parser makes a small correction, such as inserting, deleting, or replacing a token, to continue parsing.

3. Error Productions: Additional grammar rules can be used to recognize common syntax mistakes and provide more specific error messages.

4. Clear Error Reporting: The parser should indicate the unexpected token and explain what was expected at that position.

For example, the parser may display:

"Syntax error: expected an identifier or opening parenthesis after '+', but found '*'."

These techniques help the compiler identify errors clearly and continue checking the remaining program whenever possible.

5. Decision and Solution

The syntax analyzer should reject the expression a + * b because it violates the grammar rules. It should report the position of the unexpected multiplication operator and explain that an operand is required after the addition operator.

An appropriate error-recovery technique should then be applied to allow the compiler to continue analyzing the remaining input. However, the parser should not silently accept the invalid expression as a correct one.

Conclusion

The expression a + * b is syntactically invalid because two operators appear consecutively without an operand between them. Predictive and shift-reduce parsers identify this error by checking the input against the grammar rules. By using clear error messages and suitable recovery techniques, the syntax analyzer can improve error detection and help the compiler continue processing the program efficiently.

4. Scenario: Ambiguity in Conditional Statements A syntax analyzer is designed using the grammar S → if E then S | if E then S else S | a. While constructing the parsing table, conflicts arise, and the parser cannot decide which production to apply for certain inputs. In this scenario, explain how the syntax analyzer encounters ambiguity, identify the key issue known as the dangling else problem, and discuss how it impacts parsing. Suggest an appropriate modification or parsing strategy to resolve the ambiguity and ensure correct association of else clauses.

Introduction

Ambiguity in a grammar occurs when a single input string can produce more than one parse tree or more than one leftmost or rightmost derivation. In syntax analysis, ambiguity creates difficulty because the parser cannot determine a unique interpretation of the input.

In the given scenario, the grammar for conditional statements creates a problem known as the **dangling else problem**. The parser cannot determine which if statement should be associated with an else clause.

1. Analysis of the Grammar

The given grammar is:

S → if E then S

S → if E then S else S

S → a

Here, S represents a statement, E represents a condition, and a represents a simple statement.

The grammar allows conditional statements both with and without an else clause. However, when conditional statements are nested, the grammar does not clearly specify which if statement an else should belong to.

Consider the following input:

if E1 then if E2 then a else a

This statement contains two if statements but only one else clause. Therefore, the parser must decide which conditional statement is associated with the else.

2. Identification of the Dangling Else Problem

The dangling else problem occurs when an else clause can be associated with more than one unmatched if statement.

The input can have two interpretations:

Interpretation 1: Else belongs to the inner if

if E1 then (if E2 then a else a)

Here, the else clause belongs to the inner condition E2.

Interpretation 2: Else belongs to the outer if

if E1 then (if E2 then a) else a

Here, the else clause belongs to the outer condition E1.

Since both interpretations are permitted by the grammar, the same input can generate two different parse trees. This makes the grammar ambiguous.

3. Effect on Syntax Analysis

The ambiguity creates difficulty for the syntax analyzer because it cannot determine a unique parse tree for the input.

The main effects are:

1. **Parsing Conflict:** The parser may face a choice between productions with and without an else clause.
2. **Incorrect Interpretation:** Different parse trees may associate the else with different conditions.
3. **Unpredictable Program Behavior:** The selected interpretation can change which statement is executed when a condition is false.
4. **LL(1) Parsing Difficulty:** The grammar cannot be used directly as an LL(1) grammar because the alternatives can create conflicts in the predictive parsing table.

Therefore, the grammar must be modified or a suitable parsing strategy must be used to ensure a consistent interpretation.

4. Methods to Resolve the Ambiguity

A. Associate Else with the Nearest Unmatched If

The most common solution is to associate every else clause with the nearest preceding unmatched if.

Under this convention, the input:

if E1 then if E2 then a else a

is interpreted as:

if E1 then (if E2 then a else a)

This means that the else belongs to the inner if statement.

B. Use Explicit Block Delimiters

Braces or other block delimiters can make the association clear.

For example:

if (E1) { if (E2) { a; } else { a; } }

The braces clearly show that the else belongs to the inner conditional statement.

C. Rewrite the Grammar

The grammar can be rewritten by distinguishing between matched and unmatched conditional statements.

S → M | U

M → if E then M else M | a

U → if E then S | if E then M else U

Here, M represents a matched statement, while U represents an unmatched statement. This approach separates conditional statements that have complete if-else pairs from those that still have an unmatched if.

The grammar must be checked for the intended parsing properties after modification.

5. Decision and Solution

In the given scenario, the syntax analyzer should not leave the association of the else clause undefined. The grammar should be rewritten to remove ambiguity, or an appropriate parsing convention should be applied.

The recommended decision is to associate else with the nearest unmatched if. Explicit block delimiters can also be used to make nested conditional statements clearer.

If an LL(1) parser is required, the revised grammar must also be checked for FIRST/FOLLOW conflicts to ensure that its predictive parsing table contains at most one production in each cell.

Conclusion

The dangling else problem occurs because an else clause can be associated with different unmatched if statements. This ambiguity produces multiple possible parse trees and creates difficulties for syntax analysis. By associating else with the nearest unmatched if, using explicit block delimiters, or rewriting the grammar,

the ambiguity can be resolved. This ensures consistent interpretation of conditional statements and improves the reliability of parsing.

5.Scenario: Operator Precedence Conflict A parser is designed for arithmetic expressions using the grammar E → E + E | E * E | id. While parsing the input id + id * id, the syntax analyzer produces an incorrect parse tree due to ambiguity in operator precedence. Examine how the syntax analyzer processes this input, identify the issue caused by the grammar, and explain how precedence and associativity rules should be applied. Based on this, suggest what changes must be made to the grammar or parsing technique to ensure correct evaluation.

Introduction

Operator precedence determines the order in which operators are evaluated in an expression. Associativity determines how operators with the same precedence are grouped.

In the given scenario, the syntax analyzer uses the grammar E → E + E | E * E | id to parse the expression id + id * id. Since the grammar does not specify the precedence of addition and multiplication, the parser may generate an incorrect parse tree.

1. Analysis of the Grammar

The given grammar is:

E → E + E | E * E | id

Here, E represents an arithmetic expression, and id represents an identifier.

The grammar contains three productions. An expression can be formed by adding two expressions, multiplying two expressions, or using an identifier.

However, the grammar does not distinguish between addition and multiplication. Both operators appear at the same grammatical level, so the parser cannot determine which operator should be evaluated first.

Consider the input:

id + id * id

According to standard arithmetic rules, multiplication has higher precedence than addition. Therefore, the correct grouping is:

id + (id * id)

However, the given grammar also permits the grouping:

(id + id) * id

This creates ambiguity because the same input can produce two different parse trees.

2. Identification of the Operator Precedence Problem

The main issue is that the grammar does not define the relative precedence of the operators.

The syntax analyzer may construct either of the following structures:

Parse Tree 1: Multiplication First

The parser groups the second and third identifiers using multiplication and then combines the result with the first identifier using addition.

Expression:

id + (id * id)

This follows the standard arithmetic precedence rules.

Parse Tree 2: Addition First

The parser first combines the first and second identifiers using addition and then multiplies the result by the third identifier.

Expression:

(id + id) * id

This does not follow the standard precedence rules.

Therefore, the grammar is ambiguous and does not guarantee the intended parse tree.

parse tree: Correct Precedence

input Statement: id + id * id

```
        E
      / | \
    id  +  E
         / | \
       id  *  id
```

Interpretation:

id + (id * id)

Multiplication is performed first, followed by addition.

Parse Tree 2: Incorrect Precedence

```
        E
      / | \
     E  *  id
   / | \
 id  +  id
```

Interpretation:

(id + id) * id

Addition is performed before multiplication, which does not follow the usual arithmetic precedence rules.

Explanation: The original grammar allows both parse trees because it does not distinguish the precedence levels of addition and multiplication.

3. Role of Precedence and Associativity

Precedence and associativity are essential for correctly interpreting arithmetic expressions.

Operator Precedence: Multiplication has higher precedence than addition. Therefore, multiplication must be grouped before addition.

For example:

a + b * c

The correct interpretation is:

a + (b * c)

Operator Associativity: Operators with the same precedence are grouped according to their associativity. Addition and multiplication are normally left-associative in programming languages.

For example:

a * b * c

The expression is normally grouped as:

(a * b) * c

Similarly:

a + b + c

The expression is grouped as:

(a + b) + c

The grammar must represent these rules so that the syntax analyzer can construct the intended expression structure.

Operator	Precedence	Associativity	Example
Multiplication (*)	Higher	Left to right	a * b * c → (a * b) * c
Addition (+)	Lower	Left to right	a + b + c → (a + b) + c

4. Modification of the Grammar

The grammar can be modified by separating expressions, terms, and factors.

A suitable grammar is:

E → E + T | T

T → T * F | F

F → id

Here, E represents an expression, T represents a term, and F represents a factor.

The grammar gives multiplication higher precedence because multiplication is handled within the productions for T, while addition is handled within the productions for E.

When parsing:

id + id * id

the parser recognizes id * id as a term first. It then combines that term with the first identifier using addition.

Therefore, the resulting grouping is:

id + (id * id)

This grammar also represents left associativity for addition and multiplication when used with a suitable parser.

5. Making the Grammar Suitable for LL(1) Parsing

The modified grammar still contains left recursion. A conventional predictive LL(1) parser cannot directly process this left-recursive grammar.

Therefore, left recursion must be eliminated.

The equivalent grammar is:

E → T E'

E' → + T E' | ε

T → F T'

T' → * F T' | ε

F → id

This grammar separates addition and multiplication into different levels and removes left recursion.

E handles addition, while T handles multiplication. Consequently, multiplication has higher precedence than addition.

Although the grammar uses right recursion, the parser can preserve left associativity by processing operators from left to right when constructing the syntax tree or evaluating the expression.

Predictive Parsing Table

Non-terminal	**id**	**+**	*****	**$**
E	E → T E'	Error	Error	Error
E'	Error	E' → + T E'	Error	E' → ε
T	T → F T'	Error	Error	Error
T'	Error	T' → ε	T' → * F T'	T' → ε
F	F → id	Error	Error	Error

6. Decision and Solution

In the given scenario, the student should modify the grammar to represent operator precedence explicitly rather than allowing addition and multiplication at the same grammatical level.

The recommended solution is to separate expressions, terms, and factors, eliminate left recursion when LL(1) parsing is required, and reconstruct the parsing table using the revised grammar.

Alternatively, a parser that supports precedence and associativity declarations can be used, provided those rules are correctly specified.

Conclusion

The grammar E → E + E | E * E | id is ambiguous because it does not specify operator precedence. As a result, the expression id + id * id can produce multiple parse trees and may be evaluated incorrectly. By separating expressions, terms, and factors, the grammar establishes multiplication as having higher precedence than addition. Eliminating left recursion makes the revised grammar suitable for predictive LL(1) parsing. These changes ensure that the syntax analyzer constructs the correct expression structure and supports accurate evaluation.

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weigh tage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand